

راهنمای استفاده از DevFix روی Mac Intel

این فایل برای استفاده‌ی روزمره از DevFix ساخته شده تا لازم نباشد هر بار روش صحیح استفاده را به خاطر پیآوری.

وضعیت فعلی سیستم

در زمان تهیه‌ی این راهنما:

```
Mac: Intel x86_64
macOS: Monterey 12.7.6
DevFix: 2.0.4
Homebrew: 6.0.13
Homebrew Portable Ruby: 4.0.6
GitHub CLI (gh): 2.97.0 – نصب مستقیم در /usr/local/bin/gh
FFmpeg: 9.0.1-tessus – نصب مستقیم در /usr/local/bin/ffmpeg
FFprobe: 9.0.1-tessus – نصب مستقیم در /usr/local/bin/ffprobe
```

gh به GitHub لاگین شده و Credential آن در Keychain مک ذخیره شده است.

FFmpeg و FFprobe هم با تست واقعی encode/probe تأیید شده‌اند.

DevFix دقیقاً چیست؟

DevFix یک VPN سراسری برای کل مک نیست.

DevFix برای ترافیک ابزارهای توسعه ساخته شده است، مثل:

Homebrew

Git

curl

GitHub CLI

npm

Ruby installers

اجرا شود `devfix run` دیگری که با CLI و هر

DevFix در صورت نیاز Tor + Snowflake را بالا می‌آورد و یک مسیر SOCKS محلی ایجاد می‌کند.

DevFix چه کاری نمی‌کند؟

به صورت خودکار این‌ها را از تونل عبور نمی‌دهد:

Safari

Chrome

Firefox

تمام برنامه‌های مک

تمام ترافیک سیستم

یعنی روشن بودن DevFix به معنی «VPN کل سیستم» نیست.

قانون ساده‌ی روزمره

اگر اینترنت مستقیم کار می‌کند

هیچ کاری لازم نیست.

مثلاً:

```
gh auth status
```

```
git pull
```

```
curl https://github.com
```

را می‌توان اول مستقیم امتحان کرد.

اگر Homebrew، GitHub، GHCR یا دانلود Developer گیر کرد

DevFix را وصل کن:

```
devfix connect snowflake
```

صبر کن تا دقیقاً این پیام را ببینی:

Connected with built-in Snowflake.

و Shell Prompt دوباره برگردد.

بعد دستور موردنظر را از طریق DevFix اجرا کن.

اتصال و قطع اتصال

اتصال

```
devfix connect snowflake
```

وضعیت

```
devfix status
```

قطع اتصال

```
devfix disconnect
```

ری استارت مسیر

```
devfix restart
```

مهم: چه زمانی DevFix را روشن کنم؟

DevFix را وقتی روشن کن که یکی از این موارد رخ دهد:

```
timeout
TLS failure
connection reset
blocked/unreachable developer endpoint
مشکل دارد GHCR
گیر می‌کند Homebrew download
مستقیم پایدار نیست GitHub download
```

DevFix را لازم نیست همیشه روشن نگه داری.
اگر ابزار مستقیم کار می‌کند، Direct معمولاً سریع‌تر است.

Homebrew

برای Homebrew روی این مک ترجیحاً همیشه از Wrapper خود DevFix استفاده کن:

```
devfix brew update
devfix brew outdated
devfix brew install <formula>
devfix brew cleanup
devfix brew doctor
```

Update با Upgrade فرق دارد

برای تازه کردن اطلاعات Homebrew:

```
devfix brew update
```

برای دیدن پکیج‌های قدیمی:

```
devfix brew outdated
```

فقط اگر outdated چیزی نشان داد، درباره‌ی upgrade تصمیم بگیر:

```
devfix brew upgrade
```

اگر outdated خالی است، upgrade کلی لازم نیست.

هشدار macOS 12 / Tier 3

Homebrew روی Monterey ممکن است این هشدار را بدهد:

```
Warning: You are using macOS 12.  
This is a Tier 3 configuration.
```

این هشدار به تنهایی به معنی خرابی Homebrew نیست.

بعضی Formula های جدید روی Intel Monterey Bottle آماده ندارند و ممکن است مجبور به build from source شوند.

بنابراین هر خطای install لزوماً مشکل DevFix یا اینترنت نیست.

GitHub CLI — gh

gh الان مستقیم در این مسیر نصب شده:

```
/usr/local/bin/gh
```

بررسی نسخه:

```
gh --version
```

بررسی Login:

```
gh auth status
```

Login در Keychain ذخیره می شود.

خاموش کردن DevFix باعث Logout شدن gh نمی شود.

این دو مستقل اند:

gh login = Credential GitHub هویت و

DevFix = مسیر شبکه

اگر gh مستقیم کار می کند، لازم نیست DevFix روشن باشد.

اگر شبکه GitHub مشکل داشت:

```
devfix run gh <command>
```

مثلاً:

```
devfix run gh auth status
```

Git

اگر Git مستقیم کار می کند:

```
git pull
```

```
git push
```

اگر مسیر شبکه مشکل داشت:

```
devfix git pull
```

```
devfix git push
```

```
devfix git clone https://github.com/OWNER/REPO.git
```

curl و دانلود فایل

اگر URL مستقیم قابل دسترس است:

```
curl -fL -O <URL>
```

اگر مسیر مستقیم مشکل دارد:

```
devfix curl -fL -O <URL>
```

برای اتصال‌های ناپایدار می‌توان از Retry استفاده کرد:

```
devfix curl --http1.1 -fL \  
  --retry 10 \  
  --retry-all-errors \  
  --retry-delay 3 \  
  -O <URL>
```

اگر فایل کوچک با Snowflake چند ساعت ETA نشان می‌دهد، منتظر نمان. در صورت امکان Direct را امتحان کن.

اجرای هر CLI از مسیر DevFix

قالب عمومی:

```
devfix run <command> [args...]
```

مثال:

```
devfix run npm install  
devfix run python script.py  
devfix run gh repo view
```

این برای برنامه‌های Command-Line است.

آیا DevFix می‌تواند سایت یا IP فیلترشده را باز کند؟

برای CLI: بله، در حد یک درخواست شبکه

مثلاً:

```
devfix curl https://example.com
```

یا:

```
devfix curl https://1.2.3.4
```

اگر مقصد از طریق Tor/Snowflake قابل دسترس باشد، درخواست می‌تواند از آن مسیر عبور کند.

برای مرورگر: نه به صورت خودکار

DevFix در نسخه فعلی یک VPN سراسری یا Browser VPN نیست.

این دستور وجود ندارد:

```
devfix open <blocked-site>
```

و DevFix خودش Safari/Chrome را به طور خودکار Proxy نمی‌کند.

DevFix یک SOCKS محلی برای Process‌های مدیریت شده‌ی خودش می‌سازد، نه تنظیم سراسری macOS Network.

پس برای وب‌گردی عمومی از DevFix به عنوان VPN دائمی استفاده نکن.

External Proxy

اگر روزی یک Proxy خارجی معتبر داشتی، DevFix می‌تواند آن را هم استفاده کند:

```
devfix proxy set socks5h://127.0.0.1:1080
```

```
devfix connect external-proxy
```


و برای دیدن وضعیت:

```
devfix proxy status
```

پاک کردن:

```
devfix proxy clear
```

این بخش برای Proxy موجود است؛ DevFix خودش IP Proxy د نخواه تولید نمی کند.

Transportها

دیدن Transportها:

```
devfix transport list
```

وضعیت:

```
devfix transport status
```

تست Direct:

```
devfix transport test direct
```

تست Snowflake:

```
devfix transport test snowflake
```

انتخاب خودکار:

```
devfix transport auto
```

اگر Snowflake گیر کرد

اگر زیر 100٪ متوقف شد:

```
Bootstrap / Transport failure
```

اگر به 100٪ رسید ولی Endpoint validation شکست خورد:

```
Route validation failure
```

این دو مشکل متفاوت اند.

یک Session خراب Snowflake لزوماً به معنی خرابی DevFix نیست.

ابتدا:

```
devfix disconnect  
devfix connect snowflake
```

یک بار با اینترنت پایدار امتحان کن.

شبکه، Wi-Fi یا Hotspot را وسط Bootstrap یا Download عوض نکن.

Log و عیب‌یابی

آخرین Log ها:

```
devfix logs --tail 200
```

:Doctor

```
devfix doctor
```

Doctor کامل تر:

```
devfix doctor --verbose
```

قبل از فرستادن Log در جای عمومی، آن را مرور کن.

FFprobe و FFmpeg

این دو در حال حاضر مستقیم نصب شده‌اند، نه توسط Homebrew.

مسیرها:

```
/usr/local/bin/ffmpeg  
/usr/local/bin/ffprobe
```

بررسی:

```
ffmpeg -version  
ffprobe -version
```

بنابراین:

```
brew upgrade
```

آنها را آپدیت نمی‌کند.

برای آپدیت آینده باید نسخه‌ی جدید Direct به‌صورت جدا بررسی و نصب شود.

gh هم Homebrew-managed نیست

gh نیز در حال حاضر Direct install است:

```
/usr/local/bin/gh
```

پس Homebrew آن را آپدیت نمی‌کند.

قبل از نصب دوباره‌ی:

```
brew install gh  
brew install ffmpeg
```

حتماً بررسی کن که Direct install فعلی با Homebrew conflict نکند.

دستورات سریع موردنیاز

روشن کردن DevFix

```
devfix connect snowflake
```

وضعیت

```
devfix status
```

Homebrew

```
devfix brew update  
devfix brew outdated
```

GitHub CLI

```
gh auth status
```

Git از مسیر DevFix

```
devfix git <args>
```

curl از مسیر DevFix

```
devfix curl <args>
```

هر CLI

```
devfix run <command> <args>
```

خاموش کردن

```
devfix disconnect
```

چیزهایی که نباید بی دلیل انجام بدهی

بدون دلیل مشخص این کارها را انجام نده:

```
brew reinstall ...  
brew upgrade خالی است outdated وقتی  
حذف Homebrew  
حذف Portable Ruby  
disable SIP  
disable Gatekeeper  
curl -k به عنوان راه دائمی -k  
chmod 777  
backup بدون پاک کردن tor-data  
موجود Direct install روی Homebrew از gh/ffmpeg نصب دوباره
```

مدل ذهنی خیلی ساده

GitHub Login

```
|
+-- gh Credential در Keychain
|   از بین نمرود DevFix شدن disconnect ماندگار است و با
|
+-- DevFix
    فقط مسیر شبکه است
    فقط وقتی نیاز است روشنش کن
```

و:

کار می‌کند؟ Direct

```
|
+-- استفاده کن Direct → بله
|
+-- نه → devfix connect snowflake
    devfix brew / git / curl / run سپس
    devfix disconnect و بعد از پایان کار
```

قانون نهایی

DevFix را به عنوان «ابزار شبکه برای Developer CLI ها» در نظر بگیر، نه VPN دائمی کل مک.

اگر مستقیم کار می‌کند، مستقیم برو.

اگر Developer endpoint مسدود/ناپایدار است، Snowflake را روشن کن.

اگر Homebrew خطا داد، اول مشخص کن مشکل Network است یا Compatibility/Formula؛ کورکورانه reinstall یا upgrade نکن.